

Lab 15: Scene Classification

Date of the Session: ____/____/____

Time: _____

Aim: To implement a deep learning-based approach for classifying images into different scene categories using a pre-trained convolutional neural network.

Introduction: Scene classification is a fundamental problem in the field of computer vision and visual recognition, where the goal is to assign a semantic label to an entire image based on its overall content. Unlike object detection, which focuses on identifying individual objects, scene classification captures the global context of the image such as indoor, outdoor, urban, or natural environments. With the advancement of deep learning, especially Convolutional Neural Networks (CNNs), scene classification has become highly accurate and efficient. Pre-trained models like ResNet, VGG, and MobileNet are widely used as they are trained on large datasets and can generalize well to new images. This experiment demonstrates how a pre-trained CNN model can be used to classify scenes effectively.

Objectives:

- 1) To understand the concept of scene-level image classification
- 2) To explore the use of pre-trained deep learning models
- 3) To preprocess images for model input
- 4) To perform inference using a trained model
- 5) To interpret classification results

Scene Understanding: Scene classification plays a crucial role in enabling machines to understand their surroundings at a higher level. It is widely used in: Autonomous driving systems to distinguish between highways, city streets, and rural roads, Smart photo organization systems to automatically group images, Surveillance systems for environment monitoring, Robotics for navigation and decision-making. By recognizing the scene context, systems can make more intelligent and context-aware decisions.

Dataset Description: The dataset used in this experiment is the ImageNet Dataset, which is a large-scale visual database designed for image recognition tasks. It contains over 1 million images categorized into 1000 classes. Classes include objects and scenes such as *cat, dog, car, forest, building*, etc. Each image is labeled with a corresponding category. The pretrained model (ResNet-18) used in this experiment has been trained on this dataset. The dataset enables the model to learn rich visual features and generalize well to new input images.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 1

Code

```
import torch
from torchvision import models, transforms
from PIL import Image
import matplotlib.pyplot as plt
import urllib.request

classes = urllib.request.urlopen(
    "https://raw.githubusercontent.com/pytorch/hub/master/imagenet_classes.txt"
).read().decode("utf-8").split("\n")

# =====
# LOAD PRETRAINED MODEL
# =====
model = models.resnet18(pretrained=True)
model.eval()

# =====
# IMAGE TRANSFORM
# =====
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(224),
    transforms.ToTensor()
])

# =====
# SCENE CLASSIFICATION FUNCTION
# =====
def classify_scene(label):
    label = label.lower()

    # 🌿 Nature scenes
    if any(x in label for x in ["mountain", "valley", "volcano", "alp"]):
        return "Mountain Scene"
    elif any(x in label for x in ["forest", "tree", "wood"]):
        return "Forest Scene"
    elif any(x in label for x in ["sea", "ocean", "beach", "sandbar", "coast"]):
        return "Beach Scene"

    # 🏠 Indoor scenes
    elif any(x in label for x in ["bedroom", "bed"]):
        return "Bedroom Scene"
    elif any(x in label for x in ["kitchen", "dining", "table"]):
        return "Indoor Scene"
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 2

```

# 🏙️ Urban scenes
elif any(x in label for x in ["building", "street", "car", "cab", "traffic"]):
    return "Urban Scene"

# 🌳 Natural life
elif any(x in label for x in ["bird", "animal"]):
    return "Nature Scene"

else:
    return "Unknown Scene"

```

```

image_files = ["/content/beach.jpg"] # change path if needed

```

```

for img_path in image_files:
    try:
        # Load image
        img = Image.open(img_path).convert("RGB")

        # Transform image
        img_t = transform(img).unsqueeze(0)

        # Prediction
        with torch.no_grad():
            output = model(img_t)
            _, pred = torch.max(output, 1)

        # Labels
        original_label = classes[pred.item()]
        scene_label = classify_scene(original_label)

        # Display image
        plt.imshow(img)
        plt.title(f"Scene: {scene_label}")
        plt.axis("off")
        plt.show()

        # Print results
        print("Original Label:", original_label)
        print("Scene Classification:", scene_label)
        print("-" * 40)

```

```

except Exception as e:
    print(f"Error loading image {img_path}: {e}")

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 3

Output

Scene: Mountain Scene



Scene: Beach Scene



Original Label: valley

Original Label: seashore

Conclusion: In this experiment, scene classification was performed using a pretrained Convolutional Neural Network, ResNet-18. The model was able to analyze the input image and successfully predict its corresponding class label based on features learned from the ImageNet Dataset. The experiment demonstrates that pretrained CNN models can effectively extract high-level features such as textures, shapes, and patterns, enabling accurate classification of images without the need for extensive training. It also highlights the efficiency and practicality of transfer learning in solving real-world visual recognition problems. Thus, scene classification using pretrained deep learning models proves to be a powerful and reliable approach for understanding and categorizing visual data.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 4

Lab 16: Scene Graph Generation

Date of the Session: ____/____/____

Time: _____

Aim: To generate a scene graph from an input image by detecting objects and identifying the relationships between them using deep learning techniques.

Introduction : Scene Graph Generation is an advanced task in computer vision that aims to provide a detailed and structured understanding of visual scenes. Unlike traditional image classification, which assigns a single label to an image, scene graph generation identifies multiple objects within an image and captures the relationships between them. A scene graph represents an image in the form of a graph, where objects are treated as nodes and the relationships between objects are represented as edges. This structured representation helps in understanding not only *what objects are present* but also *how they interact with each other*. Scene graph generation is widely used in applications such as image captioning, visual question answering, robotics, and autonomous systems. It bridges the gap between visual data and natural language by enabling machines to interpret complex scenes more intelligently.

Objectives

- 1) To detect multiple objects present in an image
- 2) To identify relationships between the detected objects
- 3) To represent the image in the form of a structured graph
- 4) To understand interactions between objects in a scene
- 5) To enhance scene understanding beyond basic classification

Scene Understanding : Scene Graph Generation provides a deeper level of image understanding by capturing both objects and their relationships. Instead of simply identifying what is present in an image, it explains how different objects are connected and interact with each other. For example, in an image containing a person and a bicycle, the model not only detects the objects but also identifies the relationship such as “person riding bicycle.” This structured representation helps in better interpretation of complex scenes and is useful in applications like image captioning, visual question answering, and robotics.

Dataset Description: The model was trained and evaluated using the COCO dataset, which contains approximately 330,000 images. This large-scale dataset includes 80 different object categories, enabling robust object detection and recognition across a wide range of everyday items. Some of the categories include people, cars, dogs, chairs, and bottles.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 5

Code

```
import torch
import torchvision
from torchvision import transforms
import cv2
import matplotlib.pyplot as plt
img_path = "/content/img2-sgg.png" # change your image path
image_bgr = cv2.imread(img_path)
if image_bgr is None:
    print("Image not found")
    exit()

image = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)
model.eval()
COCO_CLASSES = [
    '__background__', 'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
    'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'N/A', 'stop sign',
    'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow',
    'elephant', 'bear', 'zebra', 'giraffe', 'N/A', 'backpack', 'umbrella', 'N/A',
    'N/A', 'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard',
    'sports ball', 'kite', 'baseball bat', 'baseball glove', 'skateboard',
    'surfboard', 'tennis racket', 'bottle', 'N/A', 'wine glass', 'cup', 'fork',
    'knife', 'spoon', 'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli',
    'carrot', 'hot dog', 'pizza', 'donut', 'cake', 'chair', 'couch',
    'potted plant', 'bed', 'N/A', 'dining table', 'N/A', 'N/A', 'toilet', 'N/A',
    'tv', 'laptop', 'mouse', 'remote', 'keyboard', 'cell phone', 'microwave',
    'oven', 'toaster', 'sink', 'refrigerator', 'N/A', 'book', 'clock', 'vase',
    'scissors', 'teddy bear', 'hair drier', 'toothbrush'
]

transform = transforms.Compose([
    transforms.ToTensor()
])
img_tensor = transform(image)
with torch.no_grad():
    output = model([img_tensor])[0]
boxes = output["boxes"].cpu().numpy()
labels = output["labels"].cpu().numpy()
scores = output["scores"].cpu().numpy()
threshold = 0.7
nodes = []
class_counter = {}
for obj in objects: # objects you already created
    base = obj["base_label"]
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 6

```

if base not in class_counter:
    class_counter[base] = 0
else:
    class_counter[base] += 1
node_name = base.capitalize() + "_" + str(class_counter[base])
nodes.append({
    "name": node_name,
    "base": base,
    "box": obj["box"]
})
def box_center(box):
    x1, y1, x2, y2 = box
    return ((x1+x2)/2, (y1+y2)/2)
def box_area(box):
    x1, y1, x2, y2 = box
    return abs(x2-x1) * abs(y2-y1)
def iou(boxA, boxB):
    ax1, ay1, ax2, ay2 = boxA
    bx1, by1, bx2, by2 = boxB
    ix1 = max(ax1, bx1)
    iy1 = max(ay1, by1)
    ix2 = min(ax2, bx2)
    iy2 = min(ay2, by2)
    iw = max(0, ix2 - ix1)
    ih = max(0, iy2 - iy1)
    inter = iw * ih
    areaA = box_area(boxA)
    areaB = box_area(boxB)
    union = areaA + areaB - inter
    if union == 0:
        return 0
    return inter / union

draw_img = image.copy()
for obj in objects:
    x1, y1, x2, y2 = map(int, obj["box"])
    cv2.rectangle(draw_img, (x1,y1), (x2,y2), (0,255,0), 2)
    cv2.putText(draw_img, obj["label"], (x1, y1-5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,0,0), 2)
plt.figure(figsize=(8,8))
plt.imshow(draw_img)
plt.axis("off")
plt.show()
def get_relation(boxA, boxB):
    ax1, ay1, ax2, ay2 = boxA
    bx1, by1, bx2, by2 = boxB

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 7

```

    acx = (ax1 + ax2) / 2.0
    acy = (ay1 + ay2) / 2.0
    bcx = (bx1 + bx2) / 2.0
    bcy = (by1 + by2) / 2.0
    # horizontal relation
    if acx < bcx:
        horiz = "left of"
    else:
        horiz = "right of"

    # vertical relation
    if acy < bcy:
        vert = "above"
    else:
        vert = "below"
    return horiz, vert
scene_graph = []
for i in range(len(objects)):
    for j in range(len(objects)):
        if i == j:
            continue
        obj1 = objects[i]["label"]
        box1 = objects[i]["box"]
        obj2 = objects[j]["label"]
        box2 = objects[j]["box"]
        h, v = get_relation(box1, box2)
        scene_graph.append((obj1, h, obj2))
        scene_graph.append((obj1, v, obj2))

print("\nScene Graph Triplets (subject - relation - object)\n")
for triplet in scene_graph:
    print(triplet[0], triplet[1], triplet[2])

```

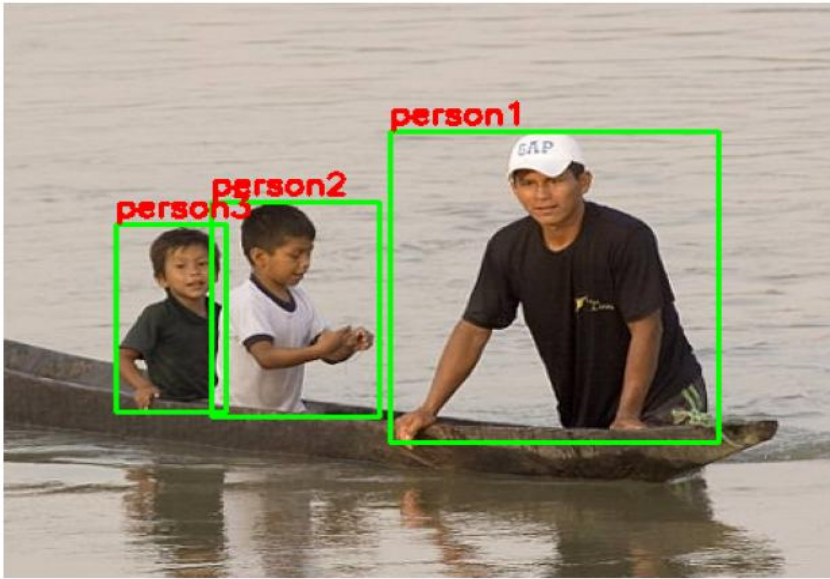
Output: Scene Graph Triplets (subject - relation - object)

```

person1 right of person2
person1 above person2
person1 right of person3
person1 above person3
person2 left of person1
person2 below person1
person2 right of person3
person2 above person3
person3 left of person1
person3 below person1
person3 left of person2
person3 below person2

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 8



Conclusion: In this experiment, scene graph generation was successfully implemented using a pretrained object detection model, Faster R-CNN. The model was able to detect multiple objects in the input image using knowledge learned from the COCO Dataset. Further, spatial relationships such as *left of*, *right of*, *above*, and *below* were computed between detected objects to form meaningful triplets in the form of (*subject – relation – object*). This allowed the image to be represented as a structured scene graph. The experiment demonstrates that combining object detection with spatial reasoning provides a deeper understanding of visual scenes compared to simple classification. Thus, scene graph generation proves to be an effective approach for advanced scene understanding and has applications in areas like image captioning, robotics, and visual reasoning.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 9

Lab 18: 3D Reconstruction Basics

Date of the Session: ____/____/____

Time: _____

Aim: To understand and implement the basic concept of 3D reconstruction using feature detection and matching between multiple images.

Introduction: 3D Reconstruction is a computer vision technique used to generate a three-dimensional representation of a scene from two-dimensional images. It helps in understanding the depth, structure, and spatial arrangement of objects in the real world. The process involves capturing multiple images of the same scene from different viewpoints and identifying corresponding points between them. Using these correspondences, the system estimates depth and reconstructs the 3D structure of the scene. Techniques such as stereo vision and feature matching are commonly used in 3D reconstruction. Feature detection algorithms identify key points in images, and matching these points across images helps in estimating depth and camera motion. This technique is widely used in applications such as robotics, virtual reality, medical imaging, and autonomous navigation. In this experiment, a basic implementation is performed using feature detection and matching to demonstrate the concept of 3D reconstruction.

Objectives:

- 1) To generate a multi-view image dataset by extracting frames from a short video sequence.
- 2) To preprocess the dataset by removing the background from each image, thereby isolating the main object of interest.
- 3) To understand the importance of image overlap and viewpoint variation in 3D reconstruction.
- 4) To detect and match feature points across multiple images using techniques such as ORB (Oriented FAST and Rotated BRIEF).
- 5) To estimate camera positions and orientations using Structure-from-Motion (SfM) methods.
- 6) To reconstruct a sparse and dense 3D model of the object from 2D images.
- 7) To analyze how preprocessing (like background removal) affects the quality and accuracy of the reconstructed 3D model.

Scene Understanding : 3D reconstruction provides a deeper understanding of a scene by estimating depth and spatial relationships between objects. Unlike 2D image analysis, it helps determine how far objects are and how they are positioned in space.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 10

Dataset Description: The dataset used in this experiment was created from a short video sequence captured using a camera. Instead of directly using images, a few seconds of video was recorded focusing on the object/scene of interest. This video was then processed to extract multiple frames at regular intervals, which serve as the input images for the 3D reconstruction pipeline. After frame extraction, background removal techniques were applied to each image to isolate the main object. This preprocessing step helps in reducing noise and unwanted features, allowing the reconstruction algorithm to focus only on the relevant object details. The final dataset consists of a set of cleaned images with minimal background, stored in PNG/JPG format. These images represent different viewpoints of the same object, ensuring sufficient variation for feature detection and matching. The extracted images maintain good overlap between consecutive frames, which is essential for accurate feature matching using algorithms like ORB (Oriented FAST and Rotated BRIEF).

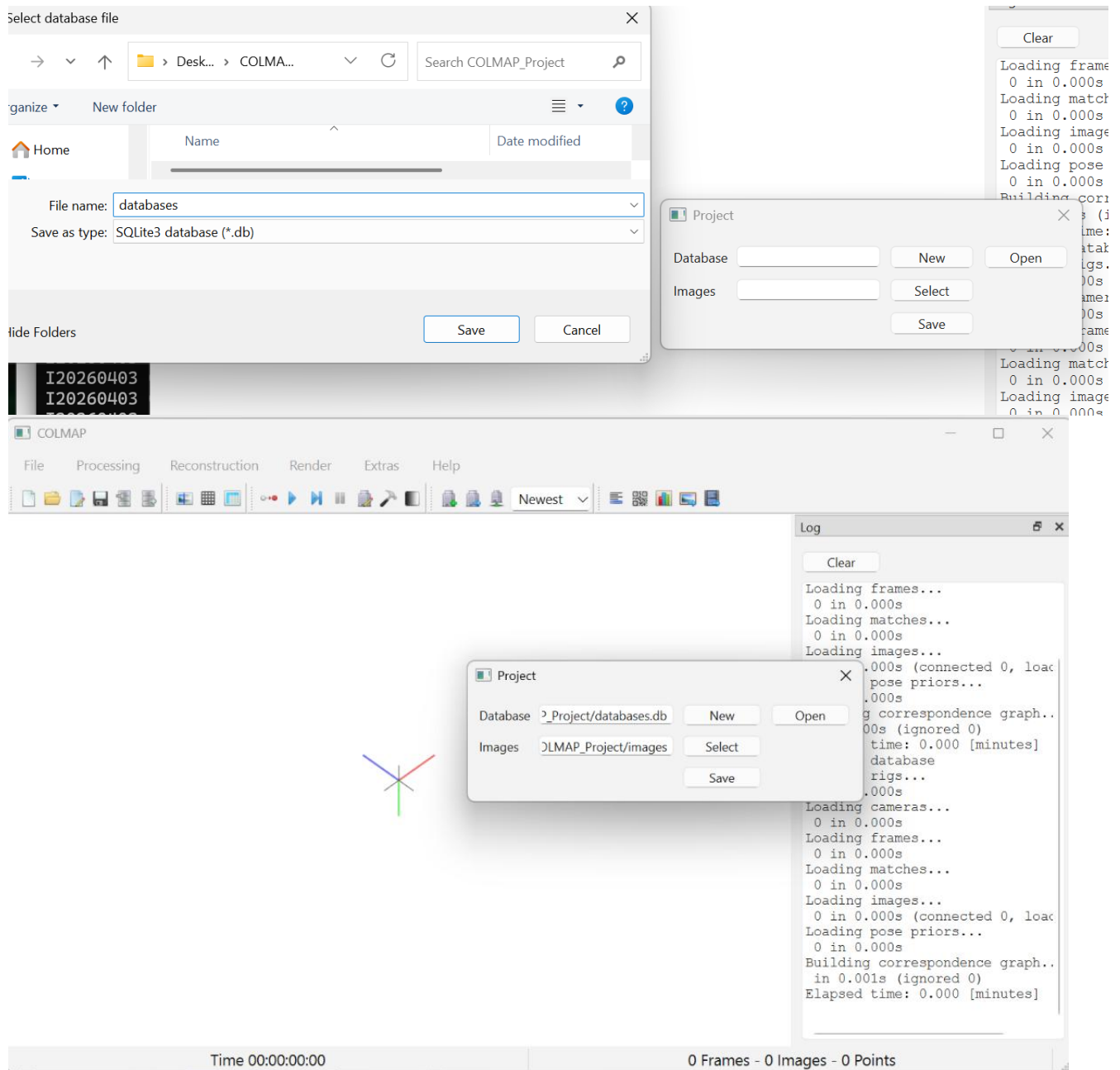
Code: Steps

A short video of the object/scene was captured using a camera to obtain multiple viewpoints.

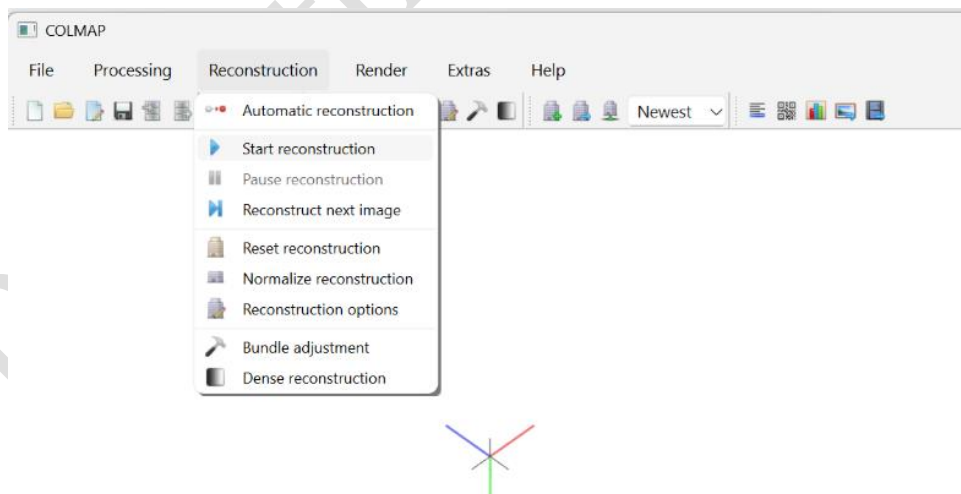
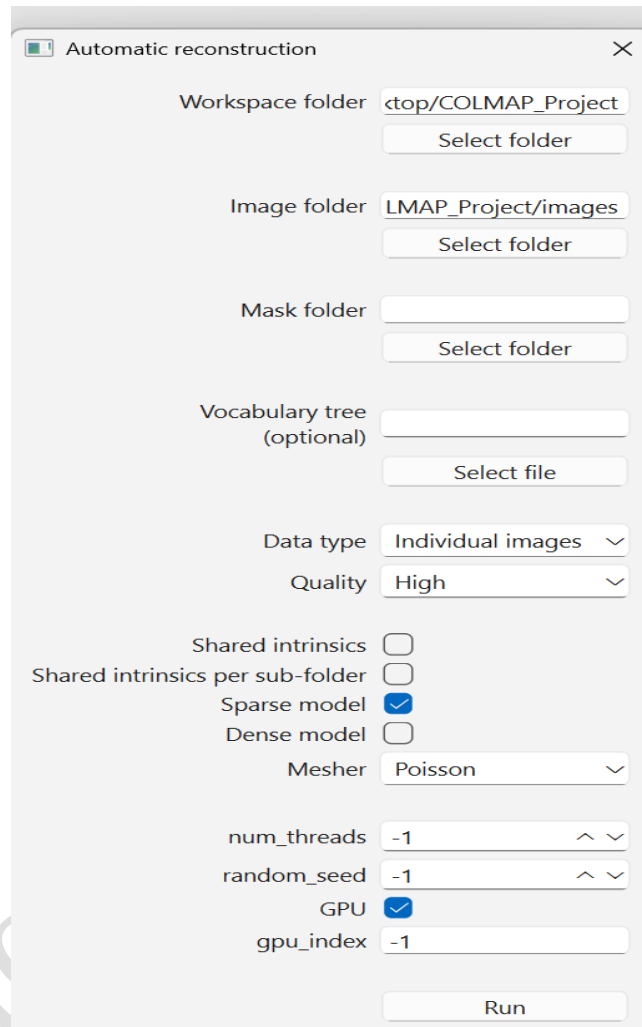
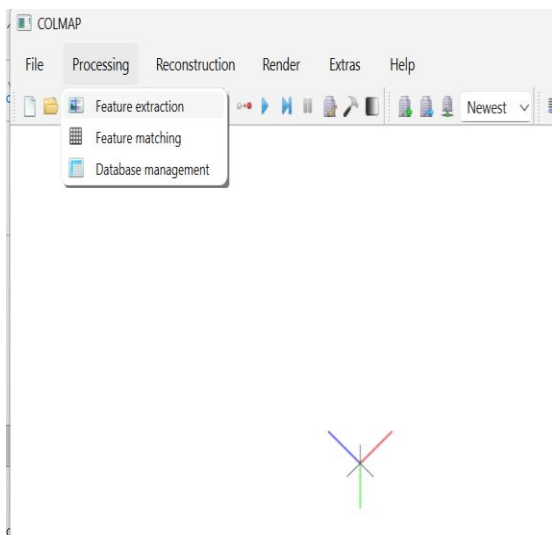
- The recorded video was processed to extract individual frames at regular intervals, generating a set of images.
- Each extracted image was pre-processed by removing the background, and a black background was applied to highlight the main object clearly.
- The processed images were organized into a folder to be used as the input dataset.
- The dataset folder was uploaded to Google Colab for further processing.
- In Colab, the reconstruction environment (COLMAP setup) was initialized, and the input image folder along with a database file was selected.
- Feature extraction was performed on all images (preprocessing step), where key points were detected using feature detection algorithms.
- Feature matching was carried out to find correspondences between images taken from different viewpoints.
- The Structure-from-Motion (SfM) process was executed to estimate camera positions and generate a sparse 3D point cloud.
- Further reconstruction steps were performed to improve the model and obtain better visualization of viewpoints.
- The final reconstructed 3D model was exported and downloaded in .PLY file format.
- The downloaded .ply file was opened in MeshLab for visualization.
 - A clear and detailed 3D reconstruction of the object was successfully obtained and analyzed.

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 11

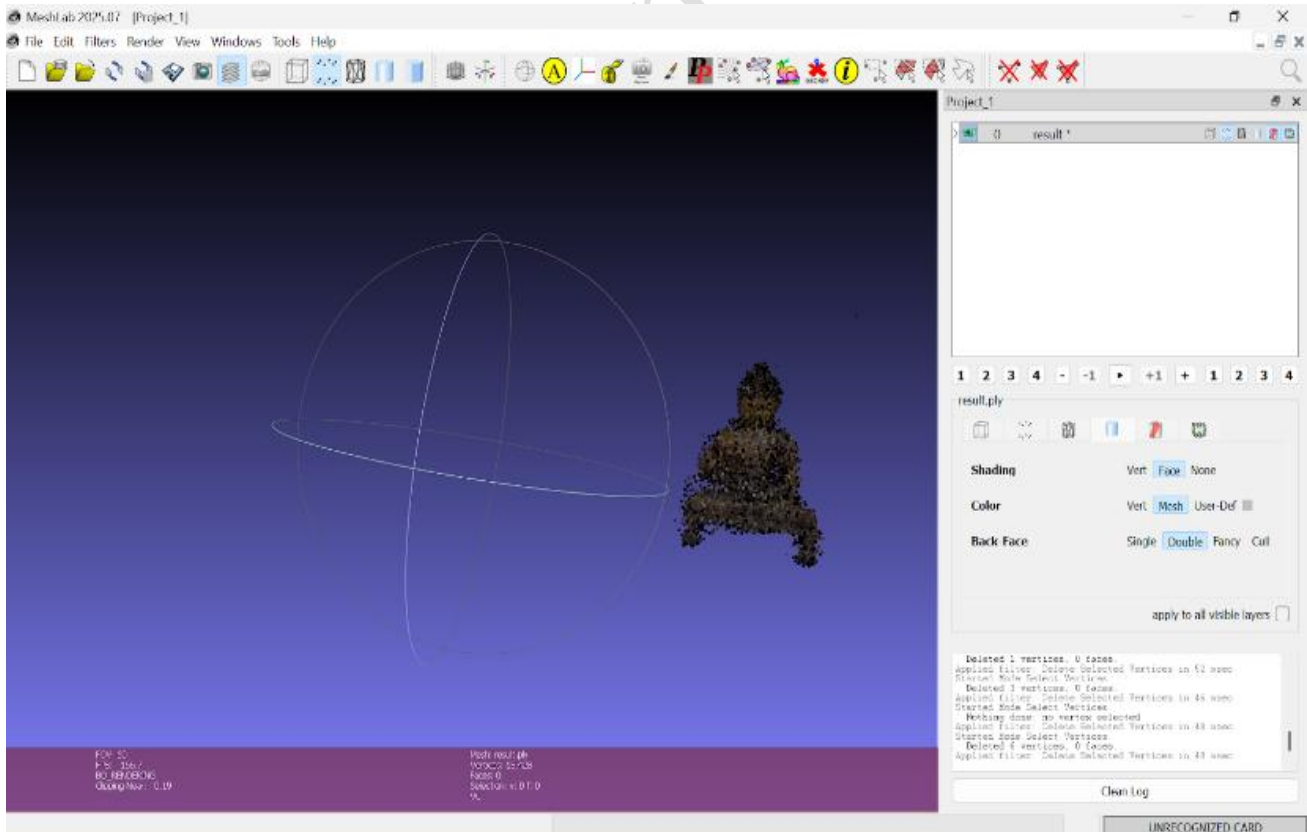
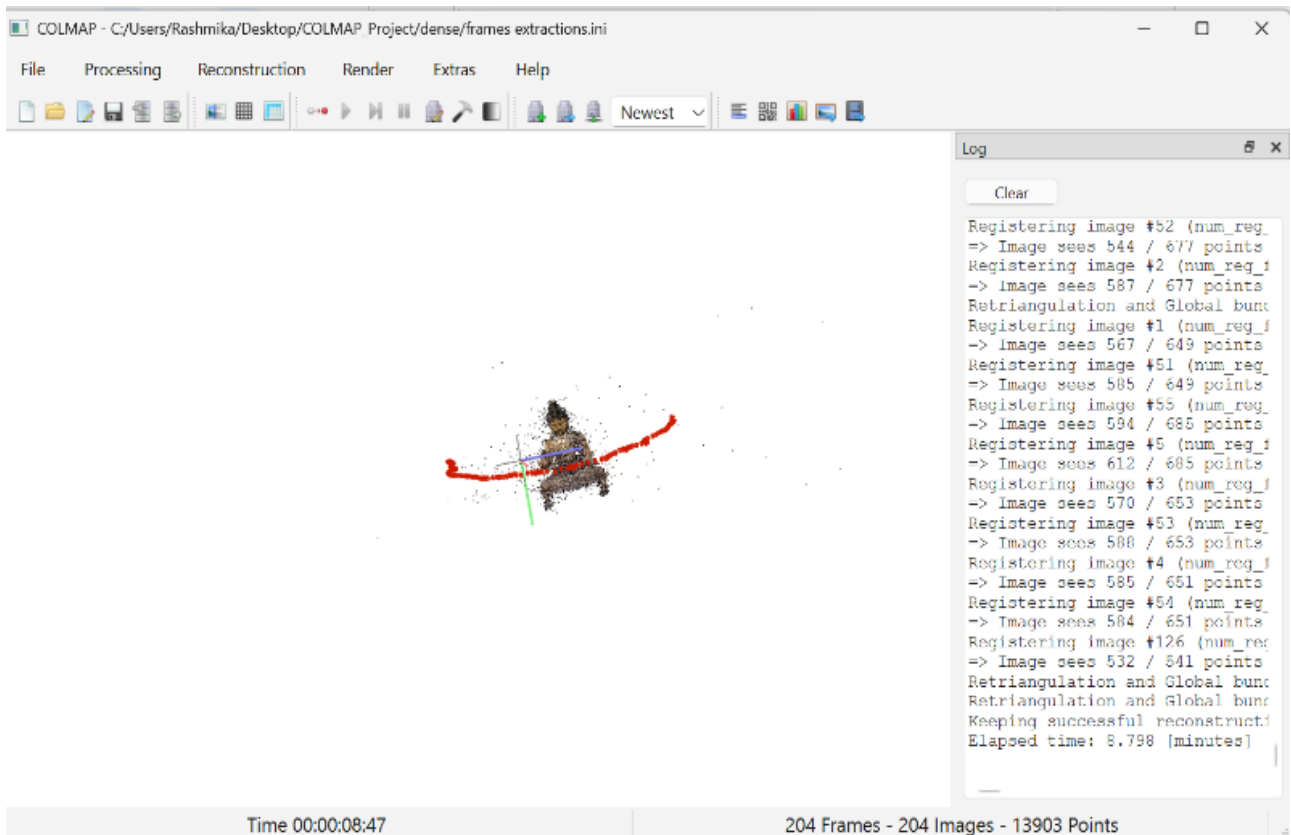
Output



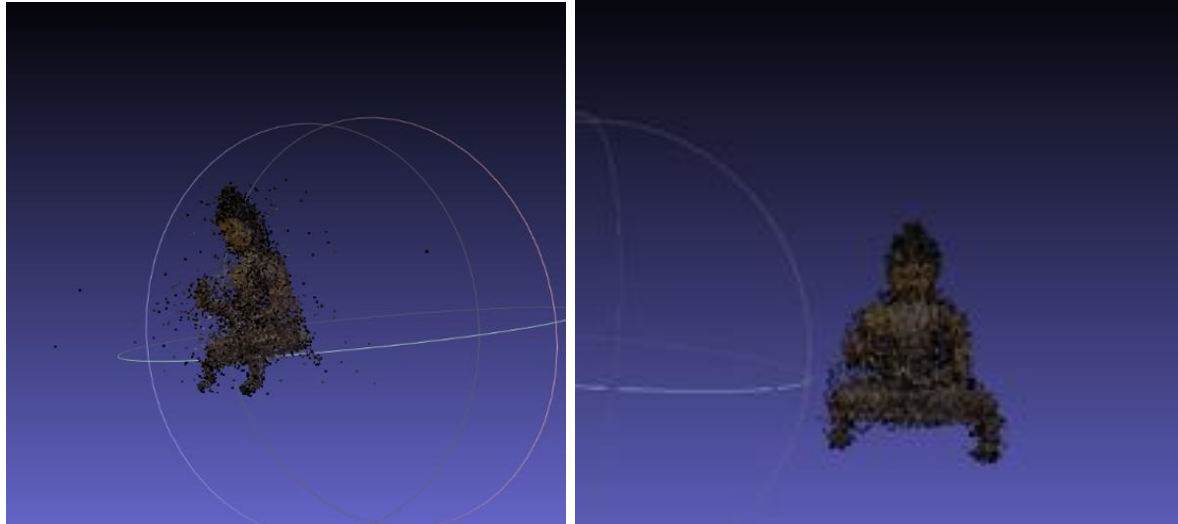
Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 12



Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 13



Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 14



Conclusion: In this experiment, a 3D reconstruction of an object was successfully achieved using a video-based dataset. A short video was captured and converted into multiple images, which were then pre-processed by removing the background and applying a black background to improve clarity. These processed images were used as input for the reconstruction pipeline. Using COLMAP in Google Colab, feature extraction and matching were performed effectively, followed by Structure-from-Motion to estimate camera positions and generate a 3D point cloud. The final model was exported as a .ply file and visualized in MeshLab, resulting in a clear and accurate 3D representation of the object. This experiment demonstrates that even a simple video can be used to create a reliable multi-view dataset for 3D reconstruction. It also highlights the importance of preprocessing steps, such as background removal and maintaining good image overlap, in improving the overall quality of the reconstructed model.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 15

Lab 19: Robotics Vision – SLAM & Object Grasping

Date of the Session: ____/____/____

Time: ____

Aim: To implement a real-time robotics vision system that performs SLAM and object detection simultaneously on a video stream.

Introduction: Robotics Vision enables robots to perceive, understand, and interact with their surroundings. In this experiment, both: SLAM (Simultaneous Localization and Mapping) → for tracking motion and environment structure Object Detection → for identifying graspable objects are applied simultaneously on a video stream. This mimics real-world robotic systems where: The robot moves and maps the environment (SLAM), While also detecting objects to interact with (grasping).

Objectives

1. Capture video input from camera/video file
2. Perform feature-based SLAM using ORB
3. Track motion between consecutive frames
4. Detect objects using Faster R-CNN
5. Overlay SLAM features and object bounding boxes
6. Visualize combined robotic perception

Application on Particular Scene : This integrated system is used in: Autonomous navigation robots, Warehouse picking robots, Self-driving cars, Surveillance robots, Industrial robotic arms.

Dataset Description: Input: Live webcam OR pre-recorded video. Model Used: Faster R-CNN (pre-trained on COCO dataset), Detects objects like: Bottle, Cup, Chair, Person.

Implementation (Code + Results)

```
import cv2
import numpy as np
import torch
import torchvision
from torchvision import transforms

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 16


```

model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)
model.to(device)
model.eval()

```

```

# COCO Labels (simplified)
COCO_CLASSES = [
    '_background_', 'person', 'bicycle', 'car', 'motorcycle', 'airplane',
    'bus', 'train', 'truck', 'boat', 'traffic light'
]
orb = cv2.ORB_create(nfeatures=1000)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

```

```

prev_kp, prev_des = None, None

```

```

cap = cv2.VideoCapture("video.mp4") # or 0 for webcam

```

```

transform = transforms.Compose([transforms.ToTensor()])

```

```

while True:
    ret, frame = cap.read()
    if not ret:
        break

```

```

# Resize for speed
frame = cv2.resize(frame, (640, 480))
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

```

```

kp, des = orb.detectAndCompute(gray, None)

```

```

if prev_des is not None and des is not None:
    matches = bf.match(prev_des, des)
    matches = sorted(matches, key=lambda x: x.distance)

```

```

for m in matches[:30]:
    pt1 = tuple(map(int, prev_kp[m.queryIdx].pt))
    pt2 = tuple(map(int, kp[m.trainIdx].pt))
    cv2.line(frame, pt1, pt2, (0,255,0), 1)

```

```

prev_kp, prev_des = kp, des

```

```

rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
tensor = transform(rgb).to(device)

```

```

with torch.no_grad():
    predictions = model([tensor])

```

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 17

```

boxes = predictions[0]['boxes']
labels = predictions[0]['labels']
scores = predictions[0]['scores']

threshold = 0.6

for box, label, score in zip(boxes, labels, scores):
    if score > threshold:
        x1, y1, x2, y2 = box.detach().cpu().numpy().astype(int)

        # Draw bounding box (Object Grasp Region)
        cv2.rectangle(frame, (x1,y1), (x2,y2), (255,0,0), 2)

        label_text = COCO_CLASSES[label.item()] if label.item() < len(COCO_CLASSES) else "obj"

        cv2.putText(frame,
                    f'{label_text}: {score:.2f}',
                    (x1, y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.5,
                    (255,0,0),
                    2)

cv2.imshow("SLAM + Object Detection", frame)

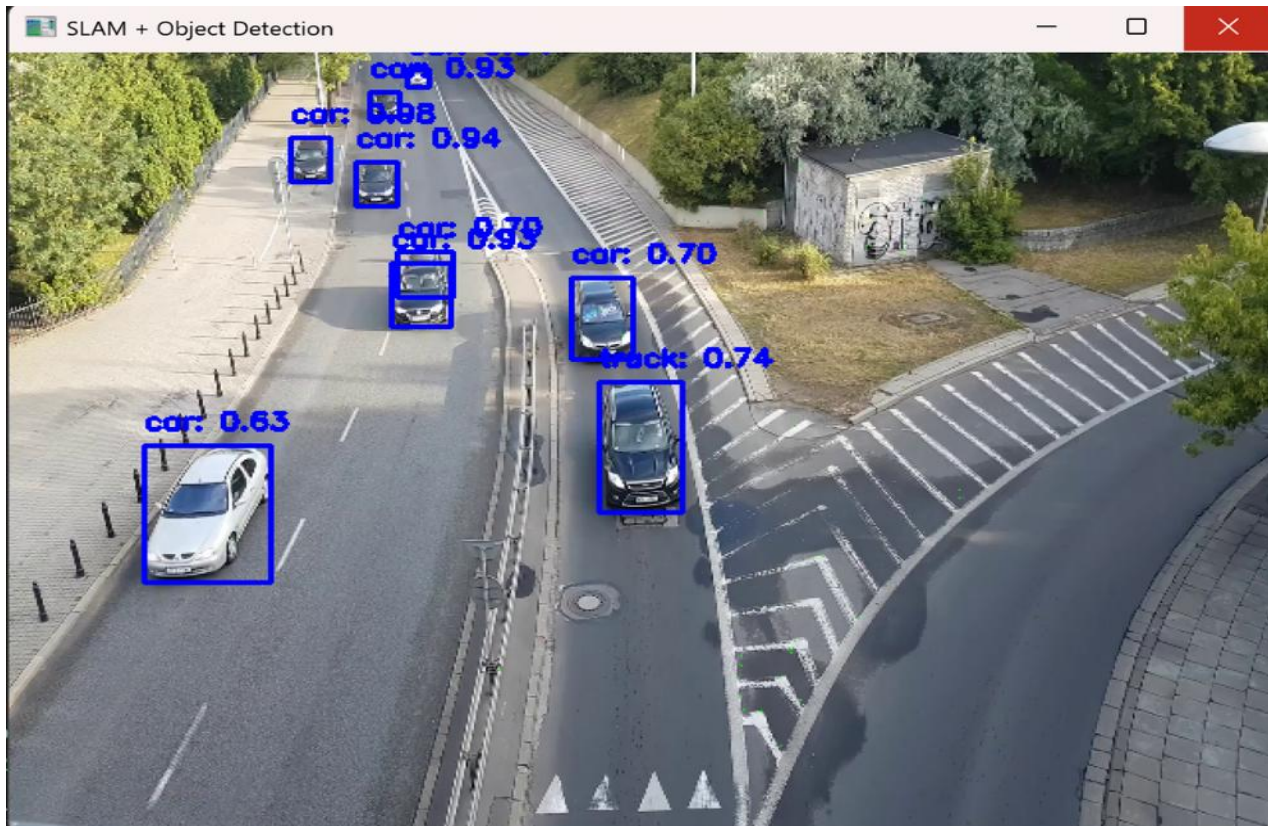
if cv2.waitKey(1) & 0xFF == 27:
    break

cap.release()
cv2.destroyAllWindows()

```

Output: SLAM Output: a) Feature points tracked across frames, b) Motion represented using green lines. **Object Detection Output:** a) Objects detected with bounding boxes, b) Labels + confidence scores displayed. **Combined Output:** Robot simultaneously: Tracks environment (SLAM), Detects graspable objects

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	P a g e 18



Conclusion: The system successfully integrates: Real-time SLAM for motion understanding, Object detection for identifying grasp targets. This demonstrates a complete robotics perception pipeline, where a robot can: Navigate its environment, Detect objects, Prepare for grasping actions.

	Marks	Date	SIGN	Comments if any by the Evaluator
Pre-Lab (10M)				
In-Lab(30M)				
Post-Lab(10M)				
TOTAL MARKS (50M)				

Course Title	Visual Recognition and Scene Understanding	ACADEMIC YEAR: 2025-26
Course Code	23SDCS19	Page 19